

Aula: Objetos Literais em JavaScript

1. Criação e Sintaxe de Objetos

O que são Objetos Literais?

Objetos literais são estruturas de dados fundamentais em JavaScript que permitem armazenar coleções de pares chave-valor. Eles são ideais para representar entidades do mundo real com múltiplas propriedades.

Sintaxe Básica



javascript

// Objeto vazio

```
const objetoVazio = {};
```

// Objeto com propriedades

```
const pessoa = {  
  nome: "Ana Silva",  
  idade: 28,  
  profissao: "Desenvolvedora",  
  ativo: true  
};
```

// Propriedades podem ter diferentes tipos de dados

```
const produto = {  
  id: 101,  
  nome: "Notebook",  
  preco: 3500.00,  
  emEstoque: true,  
  especificacoes: {  
    marca: "Dell",  
    memoria: "16GB"  
  }  
};
```

Características Importantes

- **Chaves** (propriedades): podem ser strings ou símbolos
- **Valores**: podem ser qualquer tipo de dado (strings, números, booleanos, arrays, outros objetos, funções)
- **Sintaxe**: pares chave: valor separados por vírgulas
- Objetos podem conter métodos (funções como propriedades)



javascript

```
const calculadora = {
  valor: 0,
  somar: function(num) {
    this.valor += num;
    return this.valor;
  },
  // Sintaxe moderna (ES6+)
  subtrair(num) {
    this.valor -= num;
    return this.valor;
  }
};
```

 Exercício de Fixação 1

Enunciado: Crie um objeto literal chamado `livro` com as seguintes propriedades:

- `titulo`: "1984"
- `autor`: "George Orwell"
- `ano`: 1949
- `genero`: "Ficção Distópica"
- `paginas`: 328
- `lido`: false

Solução:



javascript

```
const livro = {
  titulo: "1984",
  autor: "George Orwell",
  ano: 1949,
  genero: "Ficção Distópica",
  paginas: 328,
  lido: false
};

console.log(livro);

// Saída: { titulo: '1984', autor: 'George Orwell', ano: 1949, genero: 'Ficção Distópica', paginas: 328, lido: false }
```

2. Acesso e Modificação de Propriedades

Acessando Propriedades

Existem duas formas principais de acessar propriedades:



javascript

```
const usuario = {
  nome: "Carlos",
  email: "carlos@email.com",
  idade: 35
};
```

// 1. Notação de ponto (mais comum)

```
console.log(usuario.nome);    // "Carlos"
console.log(usuario.email);   // "carlos@email.com"
```

// 2. Notação de colchetes (útil para propriedades dinâmicas)

```
console.log(usuario["idade"]); // 35
```

// Propriedades dinâmicas

```
const propriedade = "email";
console.log(usuario[propriedade]); // "carlos@email.com"
```

Modificando Propriedades



javascript

```
const carro = {
  marca: "Toyota",
  modelo: "Corolla",
  ano: 2020,
  km: 15000
};

// Modificar propriedade existente
carro.km = 16000;
carro["ano"] = 2021;

// Adicionar nova propriedade
carro.cor = "Prata";
carro["proprietario"] = "João";

console.log(carro);
// { marca: 'Toyota', modelo: 'Corolla', ano: 2021, km: 16000, cor: 'Prata', proprietario: 'João' }
```

Deletando Propriedades



javascript

```
const produto = {
  nome: "Mouse",
  preco: 50,
  desconto: 10
};

delete produto.desconto;
console.log(produto); // { nome: 'Mouse', preco: 50 }
```

Verificando Existência de Propriedades



javascript

```
const item = {
  nome: "Teclado",
  preco: 150
};

// Operador 'in'
console.log("nome" in item); // true
console.log("estoque" in item); // false

// hasOwnProperty
console.log(item.hasOwnProperty("preco")); // true
```



Exercício de Fixação 2

Enunciado: Dado o objeto abaixo, realize as seguintes operações:

1. Acesse e exiba o nome do aluno
2. Modifique a nota para 9.5
3. Adicione uma propriedade "aprovado" com valor true
4. Delete a propriedade "turma"
5. Verifique se existe a propriedade "matricula"



javascript

```
const aluno = {
  nome: "Maria Santos",
  matricula: "2023001",
  nota: 8.5,
  turma: "A"
};
```

Solução:



javascript

```
const aluno = {
  nome: "Maria Santos",
  matricula: "2023001",
  nota: 8.5,
  turma: "A"
};

// 1. Acessar nome
console.log("Nome:", aluno.nome); // "Maria Santos"

// 2. Modificar nota
aluno.nota = 9.5;
console.log("Nova nota:", aluno.nota); // 9.5

// 3. Adicionar propriedade aprovado
aluno.aprovado = true;

// 4. Deletar turma
delete aluno.turma;

// 5. Verificar se existe matricula
console.log("Possui matrícula?", "matricula" in aluno); // true

console.log("Objeto final:", aluno);
// { nome: 'Maria Santos', matricula: '2023001', nota: 9.5, aprovado: true }
```

3. Objetos dentro de Arrays e Vice-versa

Arrays de Objetos

Arrays de objetos são extremamente úteis para representar listas de entidades similares:



javascript

```
const usuarios = [
  {
    id: 1,
    nome: "João Silva",
    email: "joao@email.com",
    ativo: true
  },
  {
    id: 2,
    nome: "Ana Costa",
    email: "ana@email.com",
    ativo: false
  },
  {
    id: 3,
    nome: "Pedro Santos",
    email: "pedro@email.com",
    ativo: true
  }
];

// Acessando dados
console.log(usuarios[0].nome);    // "João Silva"
console.log(usuarios[1].email);   // "ana@email.com"

// Iterando sobre o array
usuarios.forEach(usuario => {
  console.log(`${usuario.nome} - ${usuario.email}`);
});
```

Objetos com Arrays como Propriedades



javascript

```
const empresa = {  
  nome: "Tech Solutions",  
  fundacao: 2015,  
  funcionarios: ["Ana", "Carlos", "Beatriz"],  
  departamentos: [  
    { nome: "TI", funcionarios: 15 },  
    { nome: "RH", funcionarios: 5 },  
    { nome: "Vendas", funcionarios: 20 }  
  ],  
  tecnologias: ["JavaScript", "Python", "Java"]  
};
```

// Acessando arrays dentro de objetos

```
console.log(empresa.funcionarios[0]);      // "Ana"  
console.log(empresa.departamentos[1].nome); // "RH"  
console.log(empresa.tecnologias.length);   // 3
```

Manipulação Avançada



javascript


```
const loja = {
  nome: "Loja Virtual",
  produtos: [
    { id: 1, nome: "Mouse", preco: 50, estoque: 10 },
    { id: 2, nome: "Teclado", preco: 150, estoque: 5 },
    { id: 3, nome: "Monitor", preco: 800, estoque: 3 }
  ]
};

// Filtrar produtos em estoque
const produtosDisponiveis = loja.produtos.filter(p => p.estoque > 0);

// Encontrar produto específico
const teclado = loja.produtos.find(p => p.nome === "Teclado");

// Calcular valor total do estoque
const valorTotal = loja.produtos.reduce((total, p) => {
  return total + (p.preco * p.estoque);
}, 0);

console.log("Valor total em estoque: R$", valorTotal); // 2200
```



Exercício de Fixação 3

Enunciado: Crie um objeto escola que contenha:

- nome da escola
- um array de turmas, onde cada turma é um objeto com:
 - nome da turma
 - ano
 - array de alunos (apenas nomes)

Em seguida:

1. Adicione um novo aluno à primeira turma
2. Exiba todos os alunos da segunda turma
3. Conte o total de alunos de todas as turmas

Solução:



javascript

```
const escola = {
  nome: "Colégio Futuro Brilhante",
  turmas: [
    {
      nome: "9º Ano A",
      ano: 2025,
      alunos: ["Pedro", "Julia", "Rafael"]
    },
    {
      nome: "9º Ano B",
      ano: 2025,
      alunos: ["Camila", "Lucas", "Fernanda", "Bruno"]
    }
  ]
};

// 1. Adicionar novo aluno à primeira turma
escola.turmas[0].alunos.push("Mariana");
console.log("Primeira turma atualizada:", escola.turmas[0].alunos);

// 2. Exibir alunos da segunda turma
console.log("Alunos da segunda turma:");
escola.turmas[1].alunos.forEach(aluno => {
  console.log(`- ${aluno}`);
});

// 3. Contar total de alunos
let totalAlunos = 0;
escola.turmas.forEach(turma => {
  totalAlunos += turma.alunos.length;
});
console.log("Total de alunos na escola:", totalAlunos); // 8
```

4. Uso Prático em Modelos de Dados

Modelo: Sistema de Produtos



javascript

```
const sistemaProdutos = {
  produtos: [
    {
      id: 1,
      nome: "Smartphone XZ",
      categoria: "Eletrônicos",
      preco: 1500,
      estoque: 25,
      especificacoes: {
        marca: "TechBrand",
        memoria: "128GB",
        cor: "Preto"
      },
      avaliacoes: [
        { usuario: "João", nota: 5, comentario: "Excelente!" },
        { usuario: "Maria", nota: 4, comentario: "Muito bom" }
      ]
    },
    {
      id: 2,
      nome: "Fone Bluetooth",
      categoria: "Áudio",
      preco: 250,
      estoque: 50,
      especificacoes: {
        marca: "SoundMax",
        bateria: "20h",
        cor: "Branco"
      },
      avaliacoes: []
    }
  ],

  adicionarProduto(produto) {
    this.produtos.push(produto);
  },

  buscarPorId(id) {
    return this.produtos.find(p => p.id === id);
  },

  atualizarEstoque(id, quantidade) {
    const produto = this.buscarPorId(id);
    if (produto) {
```

```
        produto.estoque = quantidade;  
        return true;  
    }  
    return false;  
}  
};
```

Modelo: Sistema de Usuários



javascript

```
const sistemaUsuarios = {
  usuarios: [
    {
      id: 1,
      nome: "Carlos Oliveira",
      email: "carlos@email.com",
      senha: "hash123",
      perfil: {
        dataNascimento: "1990-05-15",
        telefone: "(11) 98765-4321",
        endereco: {
          rua: "Av. Principal",
          numero: 100,
          cidade: "São Paulo",
          estado: "SP"
        }
      },
      permissoes: ["leitura", "escrita"],
      ativo: true,
      dataCadastro: "2023-01-10"
    }
  ],

  cadastrar(usuario) {
    const novoUsuario = {
      ...usuario,
      id: this.usuarios.length + 1,
      dataCadastro: new Date().toISOString().split('T')[0],
      ativo: true
    };
    this.usuarios.push(novoUsuario);
    return novoUsuario;
  },

  buscarPorEmail(email) {
    return this.usuarios.find(u => u.email === email);
  },

  listarAtivos() {
    return this.usuarios.filter(u => u.ativo);
  }
};
```

Modelo: Sistema de Tarefas (To-Do List)



javascript

```
const sistemaTarefas = {
  tarefas: [
    {
      id: 1,
      titulo: "Estudar JavaScript",
      descricao: "Revisar objetos literais",
      prioridade: "alta",
      status: "em_andamento",
      dataCriacao: "2025-10-15",
      dataLimite: "2025-10-20",
      tags: ["estudos", "programação"],
      subtarefas: [
        { id: 1, titulo: "Ler documentação", concluida: true },
        { id: 2, titulo: "Fazer exercícios", concluida: false }
      ]
    },
    {
      id: 2,
      titulo: "Fazer compras",
      descricao: "Comprar itens do supermercado",
      prioridade: "media",
      status: "pendente",
      dataCriacao: "2025-10-18",
      dataLimite: "2025-10-21",
      tags: ["pessoal"],
      subtarefas: []
    }
  ],
```

```
  adicionarTarefa(tarefa) {
    const novaTarefa = {
      ...tarefa,
      id: this.tarefas.length + 1,
      status: "pendente",
      dataCriacao: new Date().toISOString().split('T')[0],
      subtarefas: []
    };
    this.tarefas.push(novaTarefa);
    return novaTarefa;
  },
```

```
  concluirTarefa(id) {
    const tarefa = this.tarefas.find(t => t.id === id);
    if (tarefa) {
```

```
        tarefa.status = "concluida";  
        return true;  
    }  
    return false;  
},  
  
listarPorPrioridade(prioridade) {  
    return this.tarefas.filter(t => t.prioridade === prioridade);  
},  
  
listarPendentes() {  
    return this.tarefas.filter(t => t.status === "pendente");  
}  
};
```



Exercício de Fixação 4

Enunciado: Usando o modelo de sistemaTarefas acima:

1. Adicione uma nova tarefa com título "Fazer exercícios físicos"
2. Marque a tarefa de id 2 como concluída
3. Liste todas as tarefas pendentes
4. Adicione uma subtarefa à tarefa de id 1

Solução:



javascript

// 1. Adicionar nova tarefa

```
const novaTarefa = sistemaTarefas.adicionarTarefa({  
  titulo: "Fazer exercícios físicos",  
  descricao: "Treino de 1 hora na academia",  
  prioridade: "alta",  
  dataLimite: "2025-10-22",  
  tags: ["saúde", "pessoal"]  
});  
console.log("Nova tarefa criada:", novaTarefa);
```

// 2. Marcar tarefa 2 como concluída

```
sistemaTarefas.concluirTarefa(2);  
console.log("Tarefa 2:", sistemaTarefas.tarefas.find(t => t.id === 2));
```

// 3. Listar tarefas pendentes

```
const tarefasPendentes = sistemaTarefas.listarPendentes();  
console.log("Tarefas pendentes:");  
tarefasPendentes.forEach(t => {  
  console.log(`- ${t.titulo} (${t.prioridade})`);  
});
```

// 4. Adicionar subtarefa

```
const tarefa1 = sistemaTarefas.tarefas.find(t => t.id === 1);  
tarefa1.subtarefas.push({  
  id: 3,  
  titulo: "Criar projeto prático",  
  concluida: false  
});  
console.log("Subtarefas da tarefa 1:", tarefa1.subtarefas);
```



Miniprojeto: Sistema de Marcação de Consultas - Clínica Veterinária

Descrição do Projeto

Criar um sistema simplificado para gerenciar consultas em uma clínica veterinária, incluindo cadastro de pets, tutores e agendamento de consultas.

Requisitos Funcionais

1. Cadastrar tutores e seus pets
2. Agendar consultas
3. Listar consultas por data
4. Listar todos os pets de um tutor
5. Cancelar consultas

Solução Completa



javascript

```
const clinicaVeterinaria = {
  nome: "Clínica Pet Vida",
  endereco: "Rua dos Animais, 123",

  tutores: [
    {
      id: 1,
      nome: "Roberto Silva",
      telefone: "(11) 98888-1111",
      email: "roberto@email.com",
      pets: [
        {
          id: 1,
          nome: "Rex",
          especie: "Cachorro",
          raca: "Labrador",
          idade: 5,
          peso: 30
        },
        {
          id: 2,
          nome: "Miau",
          especie: "Gato",
          raca: "Siamês",
          idade: 3,
          peso: 4
        }
      ]
    },
    {
      id: 2,
      nome: "Fernanda Costa",
      telefone: "(11) 97777-2222",
      email: "fernanda@email.com",
      pets: [
        {
          id: 3,
          nome: "Bolinha",
          especie: "Cachorro",
          raca: "Poodle",
          idade: 2,
          peso: 8
        }
      ]
    }
  ]
}
```

```

    }
  ],

  consultas: [
    {
      id: 1,
      tutorId: 1,
      petId: 1,
      data: "2025-10-22",
      horario: "10:00",
      veterinario: "Dra. Ana Paula",
      motivo: "Consulta de rotina",
      status: "agendada",
      observacoes: ""
    },
    {
      id: 2,
      tutorId: 2,
      petId: 3,
      data: "2025-10-22",
      horario: "14:00",
      veterinario: "Dr. Carlos Mendes",
      motivo: "Vacinação",
      status: "agendada",
      observacoes: "Trazer carteira de vacinação"
    }
  ],

```

// Método para cadastrar novo tutor

```

cadastrarTutor(nome, telefone, email) {
  const novoTutor = {
    id: this.tutores.length + 1,
    nome,
    telefone,
    email,
    pets: []
  };
  this.tutores.push(novoTutor);
  return novoTutor;
},

```

// Método para adicionar pet a um tutor

```

adicionarPet(tutorId, dadosPet) {
  const tutor = this.tutores.find(t => t.id === tutorId);

```

```
if (!tutor) {
  return { erro: "Tutor não encontrado" };
}

const novoPet = {
  id: Date.now(), // ID único baseado em timestamp
  ...dadosPet
};

tutor.pets.push(novoPet);
return novoPet;
},

// Método para agendar consulta
agendarConsulta(tutorId, petId, data, horario, veterinario, motivo, observacoes = "") {
  const tutor = this.tutores.find(t => t.id === tutorId);
  if (!tutor) {
    return { erro: "Tutor não encontrado" };
  }

  const pet = tutor.pets.find(p => p.id === petId);
  if (!pet) {
    return { erro: "Pet não encontrado" };
  }

  const novaConsulta = {
    id: this.consultas.length + 1,
    tutorId,
    petId,
    data,
    horario,
    veterinario,
    motivo,
    status: "agendada",
    observacoes
  };

  this.consultas.push(novaConsulta);
  return novaConsulta;
},

// Método para listar consultas por data
listarConsultasPorData(data) {
  return this.consultas
```

```
.filter(c => c.data === data && c.status === "agendada")
.map(consulta => {
  const tutor = this.tutores.find(t => t.id === consulta.tutorId);
  const pet = tutor.pets.find(p => p.id === consulta.petId);

  return {
    horario: consulta.horario,
    tutor: tutor.nome,
    pet: pet.nome,
    especie: pet.especie,
    veterinario: consulta.veterinario,
    motivo: consulta.motivo
  };
})
.sort((a, b) => a.horario.localeCompare(b.horario));
},
```

// Método para listar pets de um tutor

```
listarPetsPorTutor(tutorId) {
  const tutor = this.tutores.find(t => t.id === tutorId);
  if (!tutor) {
    return { erro: "Tutor não encontrado" };
  }

  return {
    tutor: tutor.nome,
    pets: tutor.pets
  };
},
```

// Método para cancelar consulta

```
cancelarConsulta(consultaId) {
  const consulta = this.consultas.find(c => c.id === consultaId);
  if (!consulta) {
    return { erro: "Consulta não encontrada" };
  }

  consulta.status = "cancelada";
  return { mensagem: "Consulta cancelada com sucesso", consulta };
},
```

// Método para buscar próximas consultas de um pet

```
proximasConsultasPet(petId) {
  const hoje = new Date().toISOString().split("T")[0];
```

```

return this.consultas
    .filter(c => c.petId === petId && c.data >= hoje && c.status === "agendada")
    .sort((a, b) => {
        if (a.data === b.data) {
            return a.horario.localeCompare(b.horario);
        }
        return a.data.localeCompare(b.data);
    });
},

// Método para gerar relatório do dia
relatorioDia(data) {
    const consultasDia = this.listarConsultasPorData(data);

    return {
        data,
        totalConsultas: consultasDia.length,
        consultas: consultasDia
    };
}
};

// ===== EXEMPLOS DE USO DO SISTEMA =====

console.log("=== CLÍNICA VETERINÁRIA - SISTEMA DE CONSULTAS ===\n");

// 1. Cadastrar novo tutor
console.log("1. CADASTRANDO NOVO TUTOR:");
const novoTutor = clinicaVeterinaria.cadastrarTutor(
    "Mariana Souza",
    "(11) 96666-3333",
    "mariana@email.com"
);
console.log(novoTutor);

// 2. Adicionar pet ao tutor
console.log("\n2. ADICIONANDO PET AO TUTOR:");
const novoPet = clinicaVeterinaria.adicionarPet(3, {
    nome: "Thor",
    especie: "Cachorro",
    raca: "Golden Retriever",
    idade: 1,
    peso: 25
});

```

```
});  
console.log(novoPet);  
  
// 3. Agendar consulta  
console.log("\n3. AGENDANDO CONSULTA:");  
const novaConsulta = clinicaVeterinaria.agendarConsulta(  
  3,  
  novoPet.id,  
  "2025-10-25",  
  "09:00",  
  "Dra. Ana Paula",  
  "Primeira consulta",  
  "Filhote, trazer histórico"  
);  
console.log(novaConsulta);
```

```
// 4. Listar consultas do dia 22/10  
console.log("\n4. CONSULTAS AGENDADAS PARA 22/10/2025:");  
const consultasDia = clinicaVeterinaria.listarConsultasPorData("2025-10-22");  
consultasDia.forEach(c => {  
  console.log(`${c.horario} - ${c.pet} (${c.especie}) - Tutor: ${c.tutor}`);  
  console.log(`  Veterinário: ${c.veterinario} - Motivo: ${c.motivo}`);  
});
```

```
// 5. Listar pets de um tutor  
console.log("\n5. PETS DO TUTOR ROBERTO SILVA:");  
const petsRoberto = clinicaVeterinaria.listarPetsPorTutor(1);  
console.log(`Tutor: ${petsRoberto.tutor}`);  
petsRoberto.pets.forEach(pet => {  
  console.log(`- ${pet.nome} (${pet.especie} - ${pet.raca}), ${pet.idade} anos`);  
});
```

```
// 6. Relatório do dia  
console.log("\n6. RELATÓRIO DO DIA 22/10/2025:");  
const relatorio = clinicaVeterinaria.relatorioDia("2025-10-22");  
console.log(`Data: ${relatorio.data}`);  
console.log(`Total de consultas: ${relatorio.totalConsultas}`);
```

```
// 7. Cancelar consulta  
console.log("\n7. CANCELANDO CONSULTA:");  
const cancelamento = clinicaVeterinaria.cancelarConsulta(1);  
console.log(cancelamento.mensagem);
```

```
// 8. Verificar consultas após cancelamento
```



```
console.log("\n8. CONSULTAS APÓS CANCELAMENTO:");
const consultasAtualizadas = clinicaVeterinaria.listarConsultasPorData("2025-10-22");
console.log(`Total de consultas ativas: ${consultasAtualizadas.length}`);
```

Explicação da Solução

Estrutura do Sistema:

- **Objeto principal** (clinicaVeterinaria): contém todos os dados e métodos
- **Array de tutores:** cada tutor possui um array de pets
- **Array de consultas:** relaciona tutores e pets através de IDs
- **Métodos:** encapsulam a lógica de negócio

Conceitos Aplicados:

1. **Objetos aninhados:** tutores contêm pets, que são objetos dentro de arrays
2. **Métodos:** funções que manipulam os dados do sistema
3. **Relacionamentos:** uso de IDs para conectar tutores, pets e consultas
4. **Validações:** verificação de existência antes de operações
5. **Transformação de dados:** métodos que retornam visualizações diferentes dos dados

Possíveis Melhorias:

- Adicionar validação de horários conflitantes
- Implementar histórico de consultas realizadas
- Adicionar controle de pagamentos
- Criar sistema de lembretes automáticos
- Adicionar prontuário médico para cada pet

Resumo da Aula

Nesta aula, você aprendeu:

✓ **Criação de objetos literais** com sintaxe de pares chave-valor ✓ **Acesso e modificação** de propriedades usando notação de ponto e colchetes ✓ **Combinação de objetos e arrays** para estruturas de dados complexas ✓ **Modelagem de dados reais** para produtos, usuários e tarefas ✓ **Desenvolvimento de um sistema completo** com métodos e relacionamentos

Os objetos literais são fundamentais em JavaScript e formam a base para conceitos mais avançados como classes, JSON, e APIs RESTful.

Continue praticando e explorando diferentes formas de estruturar seus dados!